

Πανεπιστήμιο Κρήτης - Τμήμα Επιστήμης Υπολογιστών

HY252 - Αντικειμενοστραφής Προγραμματισμός Java

Διδάσκων: Ι. Τζίτζικας - Χειμερινο Εξάμηνο 2025-2026

[PROJECT PHASE #2]

Πετράκης Ραφαήλ Ιωάννης

csd5748

02/12/25 – 02/01/26

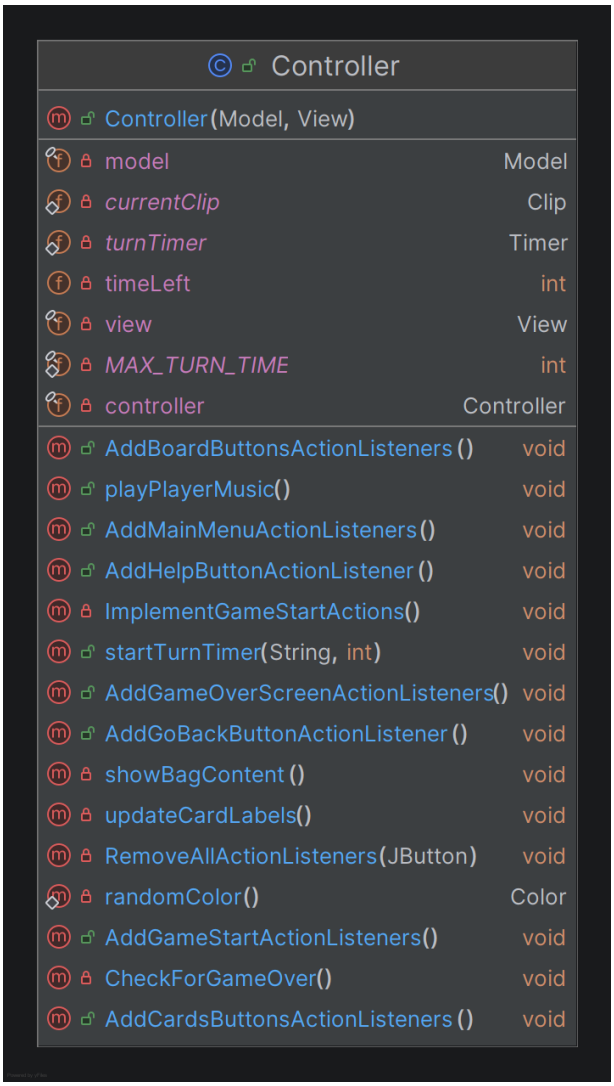
• Περιεχόμενα:

1. **Εισαγωγή**..... 2
2. **Περιγραφή του Πακέτου Controller** 2-3
3. **Περιγραφή του Πακέτου Model** 4-7
4. **Περιγραφή του Πακέτου View** 8-11
5. **Αλληλεπιδράσεις και διαγράμματα UML** 12
6. **Λειτουργικότητα Κώδικα** 13-14
7. **Συμπεράσματα και Σχόλια για την Φάση #2** 15

1. Εισαγωγή:

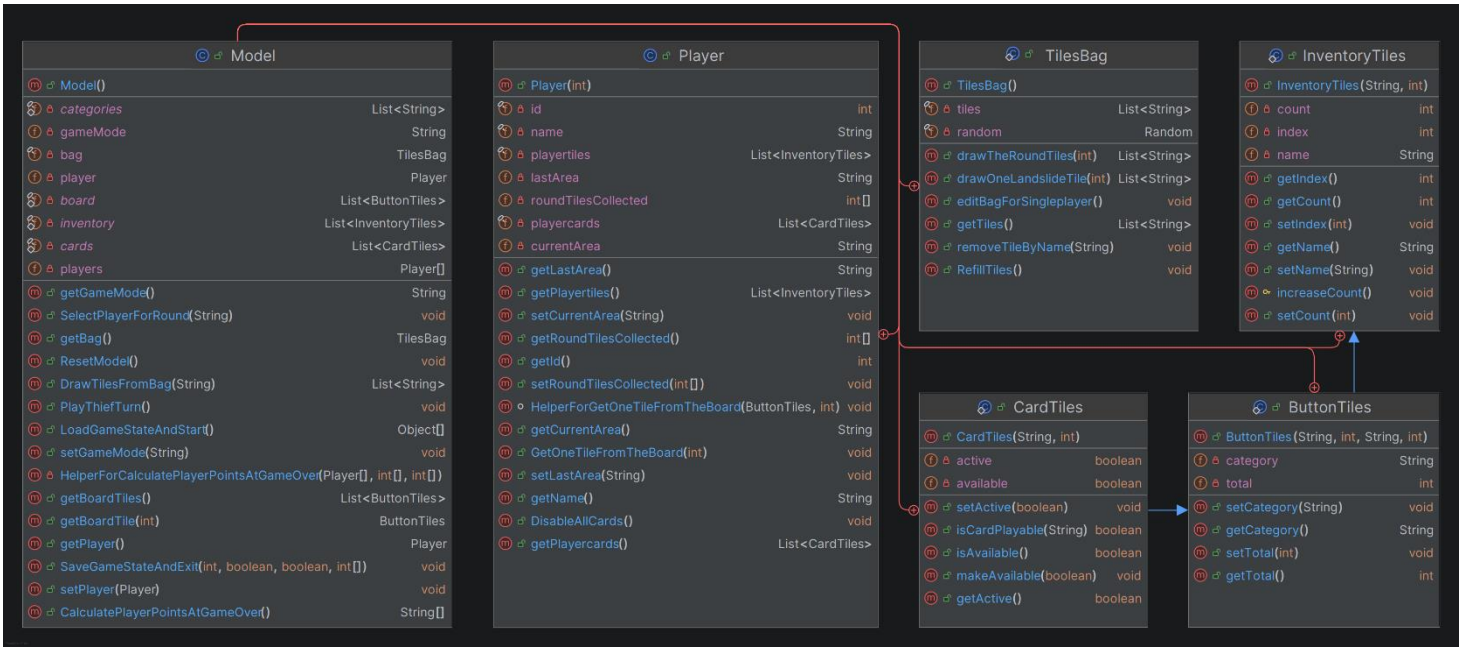
Το συγκεκριμένο Project αποτελεί την υλοποίησης του επιτραπέζιου παιχνιδιού «Amphipolis», είναι βασισμένο στο μοντέλο αρχιτεκτονικής «MVC» (Model-View-Controller) και θα δημιουργηθεί χρησιμοποιώντας την γλώσσα προγραμματισμού «Java» στο «IDE(A) IntelliJ». Έτσι, ο στόχος είναι η κλάση «Controller» να λειτουργεί ως ο εγκέφαλος του παιχνιδιού, συνδέοντας τα δεδομένα του παιχνιδιού, που βρίσκονται στο πακέτο « Model», με την γραφική διεπαφή του, η οποία βρίσκεται στο πακέτο «View». Στην συνέχεια της αναφοράς λοιπόν, θα περιγράφουνε τα (ολοκληρωμένα και τελικά) πακέτα «Model, View & Controller», τα οποία μπορούν να προσομοιώσουν το παιχνίδι …

2. Περιγραφή του Πακέτου Controller:



Αρχικά, έχουμε το UML διάγραμμα, δημιουργία του IntelliJ (Ultimate IDE) παραπάνω. Λοιπόν, έχουμε την κλάση Controller που περιέχει κάποιες από τις μεθόδους της πρώτης φάσης, αλλά υπάρχουν και αρκετές καινούργιες, άλλες απαραίτητες και άλλες βοηθητικές. Επιπλέον, έχουμε και μερικά καινούργια πεδία στην κλάση. Όπως και να έχει όμως, θα τα σχολιάσουμε όλα εδώ και τώρα. Έτσι, ας αρχίσουμε από τα πεδία. Πρώτον, το mode είναι ένα στιγμιότυπο από την επόμενη κλάση που θα παρουσιάσουμε έπειτα. Το CurrentClip είναι μια μεταβλητή που αφορά την μουσική του παιχνιδιού (υπάρχει και αντίστοιχη μέθοδος). Το TurnTimer είναι μια μεταβλητή όπου αφορά την αντίστροφη μέτρηση του κάθε γύρου κατά την διάρκεια του παιχνιδιού. Το TimeLeft είναι μια μεταβλητή που συνεργάζεται με την προηγούμενη. Το view είναι επίσης ένα στιγιότυπο από την παρά-επόμενη κλάση που θα παρουσιάσουμε στο μέλλον. Το MaxTurnTimer είναι μια μεταβλητή που σχετίζεται επίσης με το χρονόμετρο. Το controller είναι ακόμη ένα στιγμιότυπο, το οποίο δείχνει στην ίδια την κλάση μας. Τώρα, ας περάσουμε στις μεθόδους. Ο constructor: Controller(Model, View) είναι για την δημιουργία/αρχικοποίηση της κλάσης/πακέτο. Η μέθοδος AddBoardButtonsActionListeners() υπάρχει για να προσθέτει τις κατάλληλες λειτουργίες στα κουμπιά που βρίσκονται στο ταμπλό του παιχνιδιού μας. Η μέθοδος PlayPlayerMusic() χρειάζεται για να αρχίζει και να σταματάει το μουσικό υπόβαθρο του παιχνιδιού μας αναλόγως. Η μέθοδος AddMainMenuActionListeners() υπάρχει για την δημιουργία και προσθήκη λειτουργιών στα κουμπιά του αρχικού παραθύρου. Η μέθοδος AddHelpButtonActionListener() είναι μια καινούργια και μικρή, η οποία απλά συνδέει ένα συγκεκριμένο κουμπί με τον αντίστοιχο κώδικά του. Η μέθοδος ImplementGameStartActions() είναι καινούργια και απλά προστέθηκε ως μία βοηθός της προ-προηγούμενης, διότι – όπως θα δούμε και παρακάτω – κάποια από τα κουμπιά θα πρέπει να τρέξουν κάποιες παρόμοιες λειτουργίες, απλοποιώντας έτσι τον κώδικα. Η μέθοδος StartTurnTimer(String, int) ενώ δεν είναι καινούργια, πλέον δέχεται δύο ορίσματα, με σκοπό να μπορεί είτε να αρχίζει, είτε να επαναφέρετε το χρονόμετρο αντίστροφης μέτρησης του παιχνιδιού, ανάλογα. Η μέθοδος AddGameOverScreenActionListeners() υπάρχει για να προσθέτει εντολές στο κουμπί του παραθύρου που εμφανίζεται όταν τελειώσει το παιχνίδι. Η μέθοδος AddGoBackButtonActionListener() είναι καινούργια και με αρκετά όμοιο τρόπο, προσθέτει εντολές σε ένα συγκεκριμένο κουμπί. Η μέθοδος ShowBagContent() είναι νέα και δημιουργήθηκε για λόγους ελέγχου των δεδομένων, καθώς το παιχνίδι εξελίσσετε, όμως παρέμεινε επειδή τελικά είναι χρήσιμη και για άλλους λόγους – όπως θα δούμε παρακάτω. Η μέθοδος UpdateCardLabels() είναι ακόμη μια καινούργια που θα αναλύσουμε σε λίγο. Η μέθοδος RemoveAllActionListeners() είναι και αυτή νέα και χρειάζεται ώστε να μην προσθέτουμε διπλότυπες λειτουργίες στο οποιοδήποτε κουμπί κάθε φορά που αυτό πρέπει να δημιουργηθεί/εμφανιστεί, απλά αφαιρεί οποιαδήποτε παλιά πριν προστεθεί άλλη. Η μέθοδος RandomColor() δεν υπήρχε στην προηγούμενη φάση όπως και δεν θα έπρεπε να υπάρχει και σε αυτήν, αλλά υπάρχει για αισθητικούς λόγους που αφορούν τα χρώματα κουμπιών στο γραφικό περιβάλλον. Η μέθοδος AddGameStartActionListeners() είναι παλιά και απλά ενημερώνει τα κουμπιά που χρησιμοποιούνται κατά την διάρκεια του παιχνιδιού. Η μέθοδος CheckForGameOver() είναι επίσης καινούργια και, ενώ δεν χρειάζεται για κανένα λόγο, δημιουργήθηκε ώστε ως ένα από τα ζητούμενα της φάσης αυτής να είναι πιο εμφανές, απλά ελέγχει και εκτελείται αν και μόνο αν η συνθήκη τερματισμού ικανοποιήθηκε. Η μέθοδος AddCardsButtonsActionListeners() είναι για να προσθέσει (παρομοίως) λειτουργίες όταν κάποιο κουμπί κάρτας χαρακτήρα πατηθεί από κάποιον παίκτη. Έτσι, έχουν γίνει συνολικά οκτώ προσθήκες και μία αλλαγή στο συγκεκριμένο πακέτο Controller. Πιο συγκεκριμένα, από όλες αυτές, η μία αλλαγή καθώς και οι δύο προσθήκες ήταν απολύτως απαραίτητες, αλλά όπως και να έχει, η τελική μορφή θα είναι αυτή. Τέλος της κλάσης «Controller» …

3. Περιγραφή του Πακέτου Model:



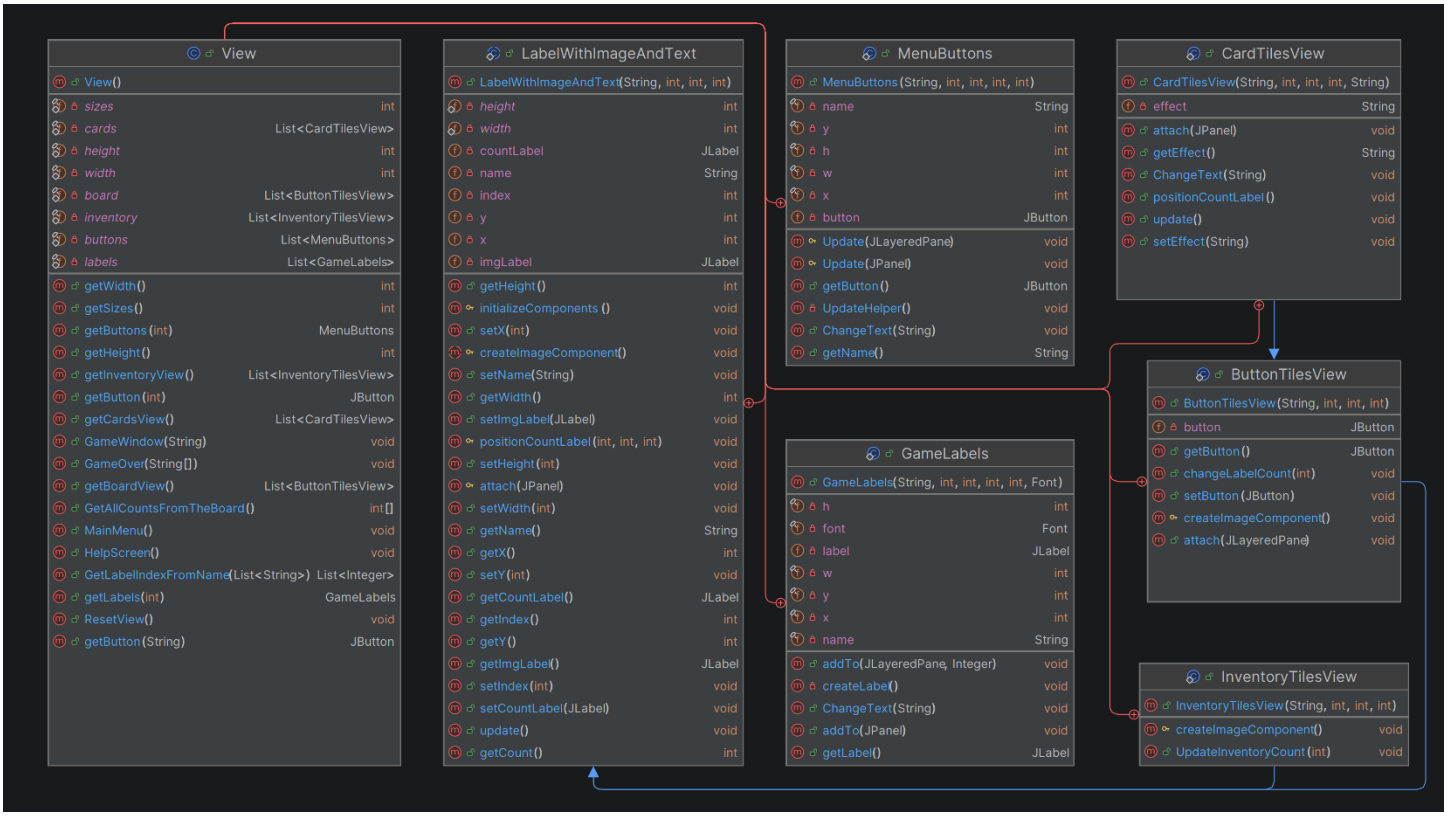
Αρχικά, έχουμε το UML διάγραμμα που δημιούργησε το IntelliJ (Ultimate IDE) παραπάνω. Λοιπόν, εδώ έχουμε την κλάση Model, η οποία δεν παρέμεινε ούτε αυτή όμοια με την πρώτη φάση. Σε γενικές γραμμές, προστέθηκαν (μόνο) αρκετά καινούργια πεδία μεταβλητών, καθώς και μεθόδοι στην κλάση, ενώ οι υπό-κλάσεις της μειώθηκαν και βελτιώθηκαν σημαντικά. Πιο συγκεκριμένα, θα αρχίσουμε από την κύρια κλάση Model. Η Categories είναι μια μεταβλητή η οποία περιέχει μια λίστα με όλες τις περιοχές που υπάρχουν στο ταμπλό του παιχνιδιού. Η GameMode είναι μια μεταβλητή η οποί περιέχει το όνομα του παιχνιδιού που είναι σε εξέλιξη (ενός ή τεσσάρων παικτών). Η Bag είναι μια μεταβλητή που προσομοιώνει την σακούλα πλακιδίων που υπάρχει στο παιχνίδι. Η Player είναι μια μεταβλητή που περιέχει το αντικείμενο του τρέχον παίκτη (πάνω στην κλάση που θα δούμε παρακάτω). Η Board είναι μια μεταβλητή που περιέχει μια λίστα με όλα τα πλακίδια που υπάρχουν στο ταμπλό σε κάθε στιγμή του παιχνιδιού. Η inventory είναι μια βοηθητική μεταβλητή που έχει δηλωθεί μια φορά και κάθε παίκτης μπορεί να πάρει ένα αντίγραφό της, καθώς αντιπροσωπεύει όλα τα πλακίδια που έχει στην κατοχή του. Η Cards είναι μια μεταβλητή που λειτουργεί με ακριβώς ίδιο τρόπο αλλά για τις κάρτες χαρακτήρα που υπάρχουν. Η Players είναι μια μεταβλητή που απλά περιέχει ένα πίνακα από όλους τους παίκτες του παιχνιδιού μας. Έτσι, πριν περάσουμε στις μεθόδους, να σημειωθεί πως καινούργια από αυτά είναι τα εξής: Categories (σε καμία περίπτωση υποχρεωτική μεταβλητή, αλλά χρήσιμη για λόγους απλότητας του κώδικα) και GameMode (υποχρεωτικό διότι παρέχουμε δύο ελάχιστα διαφορετικές τρόπους παιξίματος). Τώρα, ας περάσουμε στις μεθόδους. Η Model() μέθοδος είναι ο κονστράκτορας της κλάσης, η οποία αρχικοποιεί τα παραπάνω πεδία. Η GetGameMode() είναι μια μέθοδος για να μάθουμε τι παιχνίδι και αν εξελίσσετε. Η SelectPlayerForRound(String) είναι μια μέθοδος που ανάλογα με το όρισμα (που θα αναλύσουμε παρακάτω), διαλέγει πιος από τους παίκτες έχει σειρά να παίξει, όπως ορίζεται …

… Η GetBag() είναι μια μέθοδος που επιστρέφει την σακούλα με όλα τα πλακίδια τα οποία απομένουν μέσα της. Η ResetModel() είναι μια μέθοδος που επαναφέρει ολόκληρη την κύρια κλάση, όποτε χρειαστεί να αρχίσεις ένα νέο/διαφορετικό παιχνίδι από την αρχή. Η DrawTilesFromTheBag(String) είναι μια μέθοδος που (ομοίως) ανάλογα με το όρισμά της, αφαιρεί τα αντίστοιχα πλακίδια από την σακούλα μας. Η PlayThieveTurn() είναι μια μέθοδος που υλοποιεί τον αυτοματοποιημένο γύρο του κλέφτη στο παιχνίδι του ενός παίκτη (περισσότερες λεπτομέρειες παρακάτω). Η LoadGameStateAndStart() είναι μια μέθοδος που διαβάζει τα αποθηκευμένα δεδομένα ενός παλιού παιχνιδιού που διακόπηκε, και το συνεχίζει από το ίδιο ακριβώς σημείο. Η SetGameMode(String) είναι μια μέθοδος που απλά αποθηκεύει το παιχνίδι που είναι σε εξέλιξη. Η HelperForCalculatingPlayerPointsAtGameOver(Player[], int[], int[]) είναι μια βοηθητική μέθοδός (όπως αναφέρεται στο όνομά της) ή οποία αφορά την τελευταία μέθοδο που θα παρουσιάσουμε εδώ. Η GetBoardTiles() είναι μια μέθοδος που απλά επιστρέφει μια λίστα με όλα τα πλακίδια που υπάρχουν στο ταμπλό. Η GetBoardTile(int) είναι μια παρόμοια μέθοδος με την προηγούμενη, αλλά επιστρέφει μόνο ένα συγκεκριμένο από αυτά στην λίστα. Η GetPlayer() είναι μια μέθοδος που επιστρέφει τον παίκτη που έχει την σειρά του (τώρα). Η SaveGameStateAndExit() είναι μια μέθοδος που (ομοίως) αποθηκεύει σε ένα αρχείο (με όνομα «cookies.txt») όλα τα δεδομένα του παιχνιδιού την δεδομένη στιγμή, με σκοπό να τα χρησιμοποιήσει η παραπάνω μέθοδος αργότερα. Η SetPlayer() είναι μια μέθοδος που απλά αποθηκεύει/ενημερώνει το πιος παίκτης έχει αυτή την στιγμή την σειρά του στο παιχνίδι. Η CalculatePlayerPointsAtGameOver() είναι μια μέθοδος που, με βοήθεια, υπολογίζει τους πόντους του παιχνιδιού, καθώς και τον νικητή, όταν τελειώσει το παιχνίδι. Έτσι, πριν συνεχίσουμε, να σημειωθούν πως είναι καινούργιες προσθήκες είναι: όλες εκτός τους Getters, τους Setters, και των Save&LoadGame. Τώρα μπορούμε να περάσουμε στην πρώτη υπό-κλάση, που είναι η Player, και να παρουσιάσουμε τα (ανανεωμένα) πεδία και τις μεθόδους της. Αρχικά, η ID είναι μια μεταβλητή με το μοναδικό αριθμό του συγκεκριμένου παίκτη. Η Name είναι μια μεταβλητή με το όνομά του. Η PlayerTiles είναι μια μεταβλητή που περιέχει μια λίστα με όλα τα πλακίδια που έχει συλλέξει (από το ταμπλό) ο παίκτης (στην διάρκεια του συνολικού παιχνιδιού). Η LastArea είναι μια μεταβλητή με την περιοχή που επίλεξε ο παίκτης (από το ταμπλό) στον τελευταίο του γύρο. Η RoundTilesCollected[] είναι ένας πίνακας που αποθηκεύει πόσα πλακίδια έχει αποκτήσει ο παίκτης στον τωρινό γύρο, καθώς και από που τα απέκτησε αυτά. Η PlayerCards είναι μια μεταβλητή που περιέχει μια λίστα με όλες τις κάρτες χαρακτήρων που έχει ο παίκτης στην κατοχή του (στο παιχνίδι). Η CurrentArrea είναι μια μεταβλητή (ομοίως) με την περιοχή που επέλεξε ο παίκτης στον τωρινό γύρο του παιχνιδιού (γιατί χρειάζονται παρακάτω). Οπότε, πριν προχωρήσουμε στις μεθόδους, να σημειωθεί πως δεν άλλαξε απολύτως τίποτα εδώ. Επίσης, ακόμη δεν χρησιμοποιείτε κάποιο χρώμα που αντιστοιχεί στον κάθε παίκτη, αφού είναι ανούσιο. Έχουμε, λοιπόν, GetLastArea() που είναι μια μέθοδος που απλά επιστρέφει το αντίστοιχο πεδίο. Η GetPlayerTiles() είναι μια μέθοδος που επιστρέφει (ομοίως) την λίστα με πλακίδια του παίκτη. Η SetCurrentArea(String) είναι μια μέθοδος που απλά αποθηκεύει/ενημερώνει την τρέχουσα περιοχή τοποθέτησης στο ταμπλό. Η GetRoundTilesCollected() είναι μια μέθοδος που (ομοίως) επιστρέφει τον πίνακα με όλα τα πλακίδια που συλλέχτηκαν τον δεδομένο γύρο από τον παίκτη. Η GetId() είναι μια μέθοδος που επιστρέφει το αντίστοιχο πεδίο. Η SetRoundTilesCollected(int[]) είναι μια μέθοδος που απλά αποθηκεύει/ενημερώνει τι πλακίδια έχουν μαζευτεί από το ταμπλό αυτόν τον γύρο (με οποιοδήποτε τρόπο) …

… Η HelperForGetOneTileFromTheBoard(ButtonTiles, int) είναι μια μέθοδος που βοηθάει την παρά-επόμενη να λειτουργήσει, απλά μειώνοντας τον απαιτούμενο κώδικα. Η GetCurrentArea() είναι μια μέθοδος που (ομοίως) επιστρέφει την τωρινή περιοχή του παίκτη. Η GetOneTileFromTheBoard(int) είναι μια μέθοδος που απλά εκτελεί την λειτουργία απόκτησης ενός συγκεκριμένου πλακιδίου από το ταμπλό. Η SetLastArea(String) είναι μια μέθοδος που απλά αποθηκεύει ή ενημερώνει την περιοχή που είχε διαλέξει ο παίκτης τον τελευταίο γύρο. Η GetName() είναι μια μέθοδος που επιστρέφει το όνομα του παίκτη. Η DisableAllCards() είναι μια (βοηθητική) μέθοδος που απενεργοποιεί όλες τις κάρτες χαρακτήρα του παίκτη (παρακάτω περισσότερα). Η GetPlayerCards() είναι μια μέθοδος που (ομοίως) επιστρέφει μια λίστα με όλες τις κάρτες χαρακτήρων του παίκτη. Άρα, πριν αρχίσουμε για την επόμενη υπό-κλάση, να σημειώσουμε πως η μόνη αλλαγή εδώ είναι η προσθήκη της DisableAllCards(), που δεν είναι υποχρεωτική αφού κάλλιστα υλοποιείται με μία δομή επανάληψης οπουδήποτε. Τώρα, έχουμε την κλάση TilesBag, που είναι η σακούλα των πλακιδίων μας. Από πεδία, η Tiles είναι μια μεταβλητή που περιέχει μια λίστα με όλα τα πλακίδια που η σακούλα περιέχει ακόμα. Η Random είναι μια μεταβλητή ενός ακέραιου αριθμού, τυχαίου κάθε φορά, που χρησιμοποιείται για την επιλογή κάποιων τυχαίων πλακιδίων. Έτσι, εδώ έχουμε μόνο την προσθήκη του Random, ώστε να αρχικοποιείται μία μοναδική φορά και να χρησιμοποιείται απευθείας όποτε είναι απαραίτητος. Από μεθόδους, η TilesBag() είναι μια μέθοδος κονστράκτορας, που δημιουργεί/αρχικοποιεί την σακούλα. Η DrawTheRoundTiles(int) είναι μια μέθοδος που επιλέγει τυχαία τα πλακίδια που θα αφαιρεθούν σε κάθε γύρο. Η DrawOneLandslideTile(int) είναι μια μέθοδος που, με όμοιο τρόπο, επιλέγει τα πλακίδια που θα αφαιρεθούν αλλά μόλις επιλεχθεί ένα συγκεκριμένο, σταματάει αυτόματα. Η EditBagForSingleplayer() είναι μια μέθοδος που απλά αφαιρεί οκτώ πλακίδια συγκεκριμένου τύπου από την σακούλα (μόνο). Η GetTiles() είναι μια μέθοδος που απλά επιστρέφει το αντίστοιχο πεδίο. Η RemoveTileByName(String) είναι μια μέθοδος που απλά αφαιρεί ένα πλακίδιο από την σακούλα που έχει ένα συγκεκριμένο όνομα. Η RefillTiles() είναι μια μέθοδος που απλά ξανά γεμίζει την σακούλα με τα αρχικά πλακίδια, όταν αρχίζει ένα νέο παιχνίδι. Οπότε, να σημειωθεί σε αυτό το σημείο πως εδώ υπήρξαν αρκετές προσθήκες: πεδίο Random (παλιά το δηλώναμε κάθε φορά πριν την χρήση του), μεθόδοι DrawOneLandslideTile(int) & EditBagForSingleplayer() (είναι απαραίτητες για το παιχνίδι του ενός παίκτη που υποστηρίζουμε), και RefillTiles() (δεν είναι υποχρεωτική αλλά απλοποιεί τον κώδικα σε μεγάλο βαθμό). Τώρα, μπορούμε να μεταβούμε στην επόμενη υπό-κλάση, η οποία είναι η InventoryTiles. Αρχικά, η Count είναι μια μεταβλητή που αφορά τον αριθμό του κάθε πλακιδίου που υπάρχει. Η Index είναι μια μεταβλητή που αφορά την θέση του συγκεκριμένου πλακιδίου μέσα στην αντίστοιχη λίστα. Η Name είναι μια μεταβλητή που αφορά το όνομα του πλακιδίου. Να σημειώσουμε πως δεν έχει αλλάξει κάτι εδώ. Και, η InventoryTiles(String, int) είναι μια κονστράκτορας μέθοδος, που λειτουργεί εξίσου όμοια. Η GetIndex() είναι μια μέθοδος που επιστρέφει το αντίστοιχο πεδίο. Η GetCount είναι μια μέθοδος, ομοίως. Η SetIndex(int) είναι μια μέθοδος που αποθηκεύει την τιμή στην αρχικοποίηση. Η GetName() είναι μια μέθοδος που επιστρέφει το όνομα. Η SetName(String) είναι (ομοίως) μια μέθοδος που αποθηκεύει το όνομα. Η IncreaseCount() είναι μια μέθοδος που απλά αυξάνει τον αριθμό κατά μία μονάδα. Η SetCount(int) είναι ακόμη μια μέθοδος που ενημερώνει τον αριθμό αυτό. Έτσι, να σημειώσουμε πως δεν έχει αλλάξει κάτι ούτε εδώ. Άρα, η συγκεκριμένη υπό-κλάση παρέμεινε άθικτη. Ας πάμε και στην προ-τελευταία υπό-κλάση, στην ButtonTiles, ή οποία κάνει extends InventoryTiles (όπως έχουμε πει στην πρώτη φάση) …

… Από πεδία, η Category είναι μια μεταβλητή που δηλώνει σε πια περιοχή του ταμπλό βρίσκεται το συγκεκριμένο κουμπί. Η Total είναι μια μεταβλητή που δείχνει πόσα από αυτά τα αντίστοιχα πλακίδια μπορούν να υπάρχουν το πολύ στην σακούλα πλακιδίων (περισσότερες πληροφορίες παρακάτω). Άρα, εδώ υπήρξε μία αλλαγή: δεν χρειαζόμαστε πλέον μεταβλητή για το εάν το κουμπί είναι ενεργοποιημένο ή όχι, αλλά αυτό αντικαταστάθηκε με το μέγιστο επιτρεπτό αριθμό των αντίστοιχων πλακιδίων στην σακούλα πλακιδίων του παιχνιδιού. Και, από μεθόδους, η ButtonTiles(String, int, String, int) είναι η μέθοδος κονστράκτορας, που αρχικοποιεί όλα τα πεδία. Η SetCategory(String) είναι για να αρχικοποιεί το πεδίο. Η GetCategory() είναι μια μέθοδος που επιστρέφει το ίδιο πεδίο. Η SetTotal(int) είναι μια μέθοδος που λειτουργεί με όμοιο τρόπο, αλλά τώρα για τον αριθμό που είπαμε. Η GetTotal() είναι μια μέθοδος που τον επιστρεφει. Οπότε, και εδώ, είναι λογικό που άλλαξε λίγο η λειτουργία των δεύτερων Setter και Getter, κατά τα άλλα όλα είναι ίδια με παλιά. Έτσι, μπορούμε να περάσουμε τώρα και στην τελευταία υπό-κλάση της κύριας μας κλάσης, που είναι η CardTiles extends ButtonTiles (όπως έχουμε πει). Τα πεδία, η Active είναι μια μεταβλητή που δηλώνει εάν η συγκεκριμένη κάρτα είναι ενεργεί (στον συγκεκριμένο γύρο του παιχνιδιού) ή όχι. Η Available είναι μια μεταβλητή που δείχνει αν η συγκεκριμένη κάρτα χαρακτήρα είναι διαθέσιμη προς χρήση (γενικά για τον παίκτη) ή όχι. Άρα, υπήρξε και εδώ μια μικρή αλλαγή: το πεδίο Active αντικατέστησε το παλιό (αφού είναι απαραίτητη η συγκεκριμένη πληροφορία που παρέχει). Και, από μεθόδους, η CardTiles(String, int) είναι ο κονστρακτορας. Η SetActive(boolean) είναι μια μέθοδος που απλά ενεργοποιεί ή απενεργοποιεί την κάρτα χαρακτήρα (κάποιου παίκτη). Η IsCardPlayable() είναι μια μέθοδος που απλά επιστρέφει εάν μια κάρτα μπορεί να χρησιμοποιηθεί στην δεδομένη στιγμή του παιχνιδιού. Η IsAvailable() είναι μια μέθοδος που επιστρέφει εάν υπάρχει μια συγκεκριμένη κάρτα. Η MakeAvailable(Boolean) είναι μια μέθοδος που – ουσιαστικά – απενεργοποιεί μια κάρτα χαρακτήρα. Η GetActive() είναι μια μέθοδος που επιστρέφει αν η κάρτα είναι ενεργοποιημένη ή όχι την δεδομένη στιγμή. Οπότε, και εδώ έχουν συμβεί οι αναγκαστικές αλλαγές στους Getter και Setter λόγω της αλλαγής στο πεδίο, μόνο. Επιπλέον, οτιδήποτε άλλο υπήρχε στην παλιά έκδοση και δεν φαίνεται εδώ σημαίνει πως αφαιρέθηκε (εντελώς). Τέλος της κλάσης «Model» …

4. Περιγραφή του Πακέτου View:



Αρχικά, έχουμε το UML διάγραμμα, δημιουργία του IntelliJ (Ultimate IDE) παραπάνω. Σημείωση: οτιδήποτε δεν έχει δημιουργηθεί από εμένα εξαιρείται (πχ. Jframe, etc.) … Λοιπόν, η κλάση του View έχει υποστεί τις περισσότερες αλλαγές από την προηγούμενη φάση, και αυτό γιατί πραγματικά το χρειαζόταν. Έτσι, ας αρχίσουμε, κλασικά, από την κύρια κλάση του πακέτου, την View (extends JFrame), και τα πεδία της, προτού τις μεθόδους της και τις υπό-κλάσεις της. Η Sizes είναι μια μεταβλητή που περιέχει ένα σταθερό ακέραιο αριθμό που δηλώνει τις διαστάσεις των πάνελ που χρησιμοποιούνται για την δημιουργία των διάφορων παραθύρων. Η Cards είναι μια μεταβλητή λίστα από όλες τις κάρτες χαρακτήρων (κάθε παίκτη) που υπάρχουν στο παιχνίδι. Η Height είναι μια μεταβλητή από έναν ακόμα ακέραιο αριθμό που δηλώνει το ύψος που όλα τα παράθυρα πρέπει να έχουν. Η Width είναι (ομοίως) μια μεταβλητή από έναν διαφορετικό ακέραιο που δηλώνει το μήκος του κάθε παραθύρου (επίσης σταθερό). Η Board είναι μια άλλη μεταβλητή λίστα που περιέχει όλα τα κουμπιά που πρέπει να υπάρχουν πάνω στο ταμπλό και αντιστοιχούν στα πλακίδια (δημόσια). Η Inventory είναι μια μεταβλητή λίστα από όλα τα πλακίδια που μπορεί να έχει ένας παίκτης, άλλα με τα δεδομένα που αφορούν την συγκεκριμένη κλάση (και μόνον). Η Buttons είναι μια μεταβλητή λίστα από όλα τα κουμπιά που υπάρχουν στο παιχνίδι – γενικά σε όλα τα παράθυρα – και υπάρχει έτσι καθαρά για λόγους απλότητας και μείωση απαιτούμενου κώδικα. Η Labels είναι μια μεταβλητή ακριβώς ίδια με την προηγούμενη, όμως απευθύνεται σε διαφορετική κλάση, όπως θα δούμε στην συνέχεια, των εικόνων ή/και κειμένων που χρησιμοποιούνται …

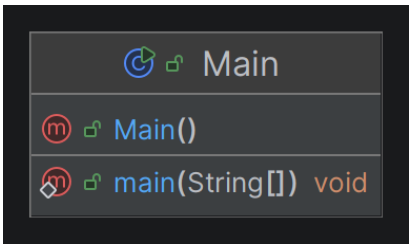
… Άρα, μέχρι στιγμής, εκτός τις διαστάσεις, όλα τα άλλα είναι καινούργια και απαραίτητα/σημαντικά. Τώρα, από τις μεθόδους, έχουμε αρχικά την View() που είναι η κονστράκτορας, υπεύθυνη για την αρχικοποίηση όλων των προαναφερόμενων. Η GetWidth() είναι μια μέθοδος που επιστρέφει την αντίστοιχη σταθερά. Η GetSizes() είναι (ομοίως) μια μέθοδος που γυρνάει τον ακέραιο αυτόν αριθμό. Η GetButtons(int) είναι μια μέθοδος η οποία επιστρέφει ένα μόνο συγκεκριμένο κουμπί από την λίστα. Η GetHeight() είναι άλλη μια μέθοδος για να δούμε το σημαντικότερο στοιχείο του παραθύρου. Η GetInventoryView() είναι μια μέθοδος που επιστρέφει ολόκληρη την λίστα με όλα τα πλακίδια που ο τρέχον παίκτης έχει αποκτήσει. Η GetButton(int) είναι μία από τις δύο μεθόδους που επιστρέφει ένα από τα κουμπιά της λίστας, αυτή με βάση την θέση του σε αυτή (τελευταία η άλλη). Η GetCardsView() είναι μια ακόμη μέθοδος μου επιστρέφει την λίστα με όλες τις κάρτες τις οποίες έχει ο τρέχον παίκτης (περισσότερες πληροφορίες παρακάτω). Η GameWindow(String) είναι μια μέθοδος που ανάλογα με το όρισμά της, δημιουργεί το αντίστοιχο παράθυρο (δηλαδή είτε για έναν παίκτη, είτε για τέσσερεις). Η GameOver(String[]) είναι μια (αντίστοιχη) μέθοδος που δημιουργεί ένα νέο παράθυρο αλλά χρησιμοποιεί το όρισμα ως δεδομένα για το παράθυρο αυτό. Η GetBoardView() είναι άλλη μια μέθοδος που (τώρα) επιστρέφει ολόκληρο την λίστα με τα κουμπιά που υπάρχουν. Η GetAllCountsFromTheBoard() είναι μια νέα μέθοδος που χρειάζεται αποκλειστικά και μόνο για όταν έρθει η σειρά του κλέφτη στο παιχνίδι ενός παίκτη. Η MainMenu() είναι η μέθοδος που απλά δημιουργεί το αρχικό παράθυρο του παιχνιδιού, με ότι αυτό συνεπάγεται. Η HelpScreen() είναι μια μέθοδος που εμφανίζει μια οθόνη βοήθειας (παρακάτω αναλυτικά). Η GetLabelIndexFromName(List<String>) είναι – ουσιαστικά – μια βοηθητική μέθοδος που θα μπορούσε να μην υπάρχει, αλλά (για ακόμα μια φορά) εξοικονομεί γραμμές κώδικα. Η GetLabels(int) είναι μια μέθοδος που επιστρέφει ένα συγκεκριμένο στοιχείο από την λίστα (πεδίο). Η ResetView() είναι μια μέθοδος που απλά επαναφέρει τα αρχικά δεδομένα πριν αρχίσει ένα νέο (οποιοδήποτε) παιχνίδι. Η GetButton(String) είναι μία παρόμοια μέθοδος που επιστρέφει ένα συγκεκριμένο κουμπί από την λίστα, αυτή την φορά όμως με βάση το όνομα του. Πριν συνεχίσουμε, να σημειώσουμε πως εκτός από τα (παλιά) Getters και Setters, και τις μεθόδους MainMenu(), GameWindow(), GameOver(), οτιδήποτε αναφέρθηκε προηγουμένως είναι καινούργια προσθήκη/επεξεργασία. Και, όλα εκτός την μέθοδο GetLabelIndexFromName() (και κάποια Getters) χρειάζονται απαραίτητα για το παιχνίδι. Συνεχίζοντας, έχουμε πρώτα την μεγαλύτερη – και σημαντικότερη – υπό-κλάση, την LabelWithImageAndText (abstract όπως είχαμε πει στην προηγούμενη φάση). Από πεδία, η Height είναι μια όμοια μεταβλητή για το ύψος της ετικέτας. Η Width είναι άλλη μια τέτοια μεταβλητή για το μήκος της. Η CountLable είναι μια μεταβλητή που θα αποθηκεύσει την ετικέτα κειμένου (για τον αριθμό). Η Name είναι μια μεταβλητή για το όνομα. Η Index είναι μια (όμοια) μεταβλητή για την θέση του στοιχείου μέσα στις λίστες που αναφέραμε. Η Y είναι μια μεταβλητή για την μια συντεταγμένη της ετικέτας. Η X είναι η άλλη μεταβλητή. Η ImageLabel είναι μια μεταβλητή που αποθηκεύει την ετικέτα εικόνας που θα χρησιμοποιούμε. Άρα, εδώ, εκτός από τα Name και Count, όλα τα άλλα είναι νέα πεδία που απαιτούνται. Και, για τις μεθόδους, η LabelWithImageAndText(String, int, int, int) είναι η μέθοδος του κονστράκτορα, που αρχικοποιεί όλα τα προαναφερόμενα πεδία. Η GetHeight() είναι μια μέθοδος που επιστρέφει το ύψος. Η InitializeComponents() είναι μια μέθοδος που ουσιαστικά αναγκάζει την εκτέλεση της παρά-επόμενης μεθόδου. Η SetX(int) είναι μια μέθοδος για την αποθήκευση του αριθμού …

… Η CreateImageComponent() είναι μια μέθοδος που απλά δημιουργεί την αντίστοιχη ετικέτα(ες) (ή και κάτι ακόμα παρακάτω). Η SetName(String) είναι μια μέθοδος που αποθηκεύει το αντίστοιχο όνομα. Η GetWIdth() είναι μια μέθοδος που επιστρέφει το μήκος. Η SetImageLabel(JLable) είναι μια μέθοδος που αποθηκεύει την ετικέτα. Η PositionCountLable(int, int, int) είναι μια μέθοδος που έχει στόχο να τοποθετήσει τις ετικέτες στο κάθε παράθυρο, σε μια συγκεκριμένη και προκαθορισμένη σταθερή θέση. Η SetHeight(int) είναι άλλη μια μέθοδος που αποθηκεύει. Η Attach(JPanel) είναι μια μέθοδο που έχει στόχο να προσθέσει την κάθε ετικέτα στο αντίστοιχο παράθυρο, ώστε να εμφανιστεί. Η SetWidht(int) είναι μια αντίστοιχη μέθοδος. Η GetName() είναι μια μέθοδος για να πάρουμε το όνομα. Η GetX() είναι μια μέθοδος για να πάρουμε την συντεταγμένη. Η SetY(int) είναι μια μέθοδος για να αποθηκευτεί ο αριθμός αυτός. Η GetCountLabel() είναι μια μέθοδος που επιστρέφει την μια από τις δύο ετικέτες μας. Η GetIndex() είναι μια μέθοδος που επιστρέφει το πεδίο. Η GetY() είναι (ομοίως) μια μέθοδος. Η GetImageLabel() είναι μια μέθοδος που επιστρέφει την άλλη από τις ετικέτες μας. Η SetIndex(int) είναι μια μέθοδος που αποθηκεύει, όπως τις άλλες. Η SetCountLabel() είναι και αυτή μια μέθοδος με παρόμοιο σκοπό. Η Update() είναι μια παλιά αλλά σημαντική μέθοδος που βοηθάει στην αρχικοποίηση/δημιουργία των ετικετών. Η GetCount() είναι και αυτή μέθοδος παραλαβής. Οπότε, εδώ, μία σημαντική προσθήκη είναι η μέθοδος GetCount(), που όπως θα δούμε παρακάτω, έχει πολύ σημαντική και έξυπνη λειτουργία. Επίσης, προστέθηκαν όλες οι Setters – Getters των νέων πεδίων. Τώρα, προχωράμε στην επόμενη (και παλιά) υπό-κλάση μας, την MenuButtons. Τα πεδία της, η Name είναι μια μεταβλητή για το όνομα, η Y είναι μια μεταβλητή για την μια συντεταγμένη, η H είναι μια μεταβλητή για το ύψος (από το height), η W είναι μια μεταβλητή για το μήκος (από το width), η X είναι μια μεταβλητή για την άλλη συντεταγμένη, και η Button είναι μια μεταβλητή για το κουμπί. Στα πεδία, λοιπόν, δεν έχουμε καμία αλλαγή. Και, για τις μεθόδους της, η (υπερφορτωμένη πλέον με βοήθεια) Update(…) είναι μια μέθοδο που δέχεται (πλέον) δύο τύπους κουμπιών (παρακάτω περαιτέρω λόγια), η GetButton() είναι μια μέθοδος που απλά επιστρέφει το κουμπί, η UpdateHelper() είναι μια μέθοδος που δεν χρειάζεται απαραίτητα, απλά αφού έχουμε δύο Update(…) τώρα, αρκετός κώδικας ανακυκλώνεται, η ChangeText(String) είναι μια μέθοδος που ενημερώνει το κείμενο του κουμπιού, και η GetName() που γυρνάει το όνομα (παλιό/νέο). Άρα, στην νέα αυτή έκδοση, όλες οι μεθόδοι είναι είτε νέες είτε επεξεργασμένες, αναγκαστικά κατά πλειοψηφία. Συνεχίζοντας, όμως, έχουμε την επόμενη (και καινούργια) υπό-κλάση GameLabels, που χρησιμοποιείται ώστε όλες (οι πολλές) ετικέτες που υπάρχουν σε ολόκληρο το παιχνίδι να είναι οργανωμένες. Αυτή, έχει για πεδία την H που είναι μια μεταβλητή για το ύψος (ομοίως), την Font που είναι μια μεταβλητή για το στυλ που θα έχει το κείμενο (εάν υπάρχει), την Label που είναι μια μεταβλητή για την ίδια την ετικέτα, την W που είναι μια μεταβλητή για το μήκος (ξανά), την Y που είναι μια ακόμη μεταβλητή για την μια συντεταγμένη, την X που είναι και η άλλη μεταβλητή, και την Name που είναι η μεταβλητή για το όνομα. Και, για μεθόδους έχουμε, την GameLabels(String, int, int, int, int, Font) που είναι ο κονστράκτορας, την AddTo(…) που είναι ακόμη μία υπερφορτωμένη μέθοδος για να προστεθεί η ετικέτα σε κάποιο παράθυρο (οποιοδήποτε και ανάλογα), την CreateLabel() που είναι μια μέθοδος σας των κουμπιών (παραπάνω), την ChangeText(String) που είναι μια όμοια μέθοδος, και την GetLabel() που είναι η μέθοδος για να πάρουμε την ίδια την ετικέτα. Σημείωση, ολόκληρη αυτή η κλάση είναι καινούργια αλλά όχι απαραίτητη, αφού …

… κάλλιστα θα μπορούσαμε να δημιουργούμε μία-μία κάθε ετικέτα στην αντίστοιχη μέθοδο της κύριας κλάσης, όμως έτσι (πιθανότατα) απλοποιείται και κερδίζουμε κώδικα. Έτσι, προχωρώντας έχουμε την (ολόιδια) υπό-κλάση InventoryTilesView extends LabelWithImageAndText, που προσομοιώνει ουσιαστικά τα πλακίδια που κάθε παίκτης έχει στην κατοχή του, καθώς και αναλαμβάνει να τα εμφανίζει στο κατάλληλο παράθυρο. Δεν διαθέτει κάποιο παραπάνω πεδίο από αυτά που κληρονομεί, αλλά έχει κάποιες μεθόδους. Η InventoryTilesView(String, int, int, int) είναι ο κονστράκτορας της κλάσης, η CreateImageComponent() είναι μια μέθοδος παρόμοια με την παραπάνω (που αναφέραμε), και η UpdateInventoryCount(int) είναι μια μέθοδος που ενημερώνει τον αριθμό που πρέπει να εμφανίζεται στην ετικέτα. Άρα, αυτή η κλάση έχει παραμείνει όπως ήταν (στην προηγούμενη φάση). Παρακάτω έχουμε την υπό-κλάση ButtonTilesView extends LabelWithImageAndText, η οποία φαίνεται και όντως είναι σχεδόν ίδια με την προηγούμενη, απλά για κουμπιά. Πιο συγκεκριμένα, εδώ υπάρχει μόνο μια παραπάνω μεταβλητή (πεδίο) Button, που είναι το κουμπί. Και, από μεθόδους, η ButtonTilesView(String, int, int, int) είναι ο κονστράκτορας, η GetButton() είναι μια μέθοδος που επιστρέφει αυτό το κουμπί, η ChangeLabelCount() είναι μια μέθοδος που (επίσης) αλλάζει τον αριθμό που πρέπει να εκτυπωθεί στο παράθυρο. Η SetButton(JButton) είναι μια μέθοδος που αποθηκεύει/αρχικοποιεί το κουμπί ως πεδίο, η CreateImageComponent() είναι (και αυτή) μια μέθοδος που δημιουργεί την αντίστοιχη εικόνα, και η Attach(…) είναι μια μέθοδος που απλά και εύκολα προσθέτει το κουμπί στο παράθυρο. Οπότε, η συγκεκριμένη κλάση έχει υποστεί μία (μόνον) αλλαγή: αφαιρέθηκε το πεδίο ActionListener, αφού δεν πρέπει και χρειάζεται να αποθηκεύεται κάτι τέτοιο, τελικά. Επιπλέον, έχουμε και την τελευταία υπό-κλάση CardTilesView extends ButtonTilesView (όπως είχαμε αναφέρει ξανά), και είναι υπεύθυνη για τις κάρτες χαρακτήρων που υπάρχουν στο παιχνίδι μας. Το πεδίο που έχει (εκτός τα άλλα) είναι η Effect μεταβλητή, η οποία απλά περιέχει την δυνατότητα που παρέχει η συγκεκριμένη κάρτα. Να σημειωθεί πως αυτή είναι νέα προσθήκη, η οποία αντικατέστησε το παλιό πεδίο Available (που είπαμε παραπάνω τον λόγο). Οι μεθόδοι που υπάρχουν είναι – αρχικά – η CardTilesView(String, int, int, int, String) που χρησιμεύει ως ο κονστράκτορας της κλάσης. Η Attach(…) είναι μια μέθοδος που απλά προσθέτει και εμφανίζει την κάρτα αυτή στο παράθυρο που πρέπει. Η GetEffect() είναι μια μέθοδος που απλά επιστρέφει το πεδίο. Η ChangeText(String) είναι μια μέθοδος που αλλάζει την περιγραφή (κείμενο ετικέτας) της κάρτας-κουμπί και χρειάζεται αφού η (υπό)κλάση μας κληρονομεί. Η PositionCountLabel() είναι μια (όμοια) μέθοδος που έχει σκοπό να προσαρμόσει την ετικέτα και το κουμπί που παρέχει η κλάση σε οποιοδήποτε παράθυρο χρειαστεί. Η Update() είναι (Και αυτή) μια μέθοδος που βοηθάει στην αρχικοποίηση όλων των πεδίων της κλάσης, όταν αυτή καλεστεί για πρώτη φορά. Η SetEffect(String) είναι μια μέθοδος που απλά αποθηκεύει το (κείμενο) της δύναμης που έχει η κάθε κάρτα χαρακτήρα. Άρα, εδώ υπάρχουν δύο νέες προσθήκες (εκτός από της προηγούμενη αντικατάσταση, και με ότι αυτή συνεπάγεται), η ChangeText(String) και η PositionCountLabel() μέθοδος, και αυτό γιατί δεν είχε αξιολογηθεί ορθός η υλοποίηση της συγκεκριμένης υπό-κλάσης στην προηγούμενη φάση/έκδοση. Και με αυτό, τελειώσαμε και την παρουσίαση του τελευταίου μας πακέτου, του «View», το οποίο πράγματι είχε αρκετές αλλά και σημαντικές τροποποιήσεις, ώστε το παιχνίδι να μπορέσει να δημιουργηθεί όσο δυνατόν καλύτερο …

5. Αλληλεπιδράσεις και διαγράμματα UML:

Αρχικά, όλα τα UML διαγράμματα παρουσιάστηκαν στις τρεις παραπάνω ενότητες αναλυτικά, οπότε δεν υπάρχει λόγος να ξανά προστεθούν εδώ. Λοιπόν, σύμφωνα με όσα ειπώθηκαν προηγουμένως, έτσι είναι το παιχνίδι που έχει δημιουργηθεί και ολοκληρωθεί. Άρα, η λογική του μοντέλου MVC έχει παραμείνει όπως πρέπει. Επίσης, όπως είχαμε τονίσει και στην προηγούμενη αναφορά, για να επικοινωνήσουν μεταξύ τους τα τρία πακέτα μας, χρησιμοποιείται ο Controller. Τα πακέτα Model & View δεν γνωρίζουν το ένα το άλλο και δεν επικοινωνούν μεταξύ τους. Επίσης, οι κονστράκτορες των πακέτων αυτών απλά αρχικοποιούν όλα τα απαραίτητα δεδομένα, δεν εκτελούν τίποτα από μόνα τους. Με λίγα λόγια, το Model δίνει και παίρνει από και στον Controller, ο Controller δίνει και παίρνει από και προς το View. Σημείωση: υπάρχει και ένα τέταρτο πακέτο που είναι η κλάση Main, η οποία θα αναλυθεί στην παρακάτω ενότητα (μαζί με άλλα πράγματα), όπως φαίνεται και στην παρακάτω εικόνα, που είναι το διάγραμμα UML της. Προς το παρόν, αυτή απλά δημιουργεί ένα στιγμιότυπο από κάθε ένα άλλο πακέτο …



6. Λειτουργικότητα Κώδικα:

Λοιπόν, σε αυτή την ενότητα της συγκεκριμένης αναφοράς (Ζητούμενο: Αναφορά Β' Φάσης) θα εξηγηθεί πως δουλεύει ο κώδικας μας, και ουσιαστικά το παιχνίδι μας, αλλά χωρίς να εναποθέτουν αντίστοιχα τμήματα αυτούσιου κώδικα – μόνο η λογική. Πριν από αυτό, σημαντική προϋπόθεση είναι οι παραπάνω ενότητες, που επεξηγούν ένα-ένα τα πακέτα που χρησιμοποιούνται, μαζί με τις αντίστοιχες εικόνες (Ζητούμενο: UML διαγράμματα). Όπως είναι προφανές, το παιχνίδι μπορεί να ξεκινήσει είτε από το εκτελέσιμο αρχείο που έχουμε δημιουργήσει (Ζητούμενο: ".jar"), είτε από την Main() μέθοδος στο αρχείο/πακέτο Main.java. Όπως και να έχει όμως, η Main() θα εκτελεστεί αρχικά. Αυτή, απλά δημιουργεί τρία αντικείμενα με την εξής σειρά: Model, View, Controller. Τώρα, μόλις δημιουργηθεί το πρώτο, Model, εφόσον ο Constructor – όπως είδαμε – είναι κενός, απλά αρχικοποιούνται όλα τα πεδία που είδαμε παραπάνω (Ζητούμενο: Αρχικοποίηση Παικτών & Σακούλας Πλακιδίων). Το ίδιο ακριβώς ισχύει και συμβαίνει και με το View (Ζητούμενο: Αρχικοποίηση Ταμπλό). Ακόμη, το ίδιο αλλά και κάτι παραπάνω γίνεται με τον Controller, αφού αυτός έχει Constructor μέθοδο, η οποία λέει στο View να εκτελέσει την μέθοδο MainMenu(), και έπειτα εκτελεί την δική του μέθοδο AddMainMenuActionListeners(). Τώρα, η πρώτη, απλά δημιουργεί και εμφανίζει το αρχικό παράθυρο, που περιέχει μια εικόνα και τέσσερα κουμπιά: παιχνίδι για 4 παίκτες, παιχνίδι για 1 παίκτη (Ζητούμενο: One Player Game), συνέχιση αποθηκευμένου παιχνιδιού – αν υπάρχει – και κουμπί βοήθειας. Από την άλλη, η δεύτερη είναι βοηθητική μέθοδος που προσθέτει στα συγκεκριμένα κουμπιά λειτουργίες, ώστε όταν πατηθούν – από τον χρήστη – να συμβεί κάτι συγκεκριμένο. Αυτή την στιγμή, λοιπόν, βρισκόμαστε στο αρχικό μενού και δεν συμβαίνει τίποτα μέχρι να πατήσουμε κάποιο από τα κουμπιά. Οπότε, ας αρχίσουμε με το τελευταίο και ευκολότερο κουμπί βοήθειας, που όταν πατηθεί, τρέχει η μέθοδος AddHelpButtonActionListener() του Controller, που απλά καλεί την HelpScreen() του View, και την AddGoBackButtonActionListener() του Controller. Η πρώτη, απλά αλλάζει/δημιουργεί ένα νέο παράθυρο, το οποίο περιέχει μια εικόνα σε JLabel με βοηθητικές πληροφορίες για τον χρήστη, καθώς και ένα κουμπί επιστροφής, ώστε να μπορούμε να γυρίσουμε πίσω στο αρχικό μενού μας. Και, λογικά, η δεύτερη απλά ενισχύει το συγκεκριμένο κουμπί με την δυνατότητα να εκτελεστούν ξανά οι ίδιες με προηγουμένως μεθόδοι MainMenu() και AddMainMenuActionListeners(). Στην συνέχεια, ας δούμε το κουμπί για παιχνίδι τεσσάρων παικτών, που μόλις πατηθεί γίνονται αρκετά πράγματα με την σειρά. Αρχικά, εκτελούμε την μέθοδο του Model SetGameMode(…), έπειτα την SelectPlayerForRound(...), και τέλος την ImplementGameStartActions() του Controller. Η πρώτη, απλά ενημερώνει πως πρόκειται να αρχίσει το συγκεκριμένο παιχνίδι. Η δεύτερη, τώρα, ανάλογα με το όρισμα που θα δώσει ο Controller, είτε θα δημιουργήσει και θα αρχικοποιήσει του παίκτες μας, με βάση την υπό-κλάση Player του Model, εάν δεν υπάρχουν ήδη, αλλιώς θα δώσει σειρά να παίξει ο επόμενος παίκτης από αυτούς που υπάρχουν (Ζητούμενο: Τήρηση Σειράς). Τέλος, η τρίτη και μεγαλύτερη μέθοδος, απλά θα ζητήσει από το View να εκτελέσει την μέθοδο GameWindow(…), που θα εξηγήσουμε στην επόμενη πρόταση, από τον Controller τις AddGameStartActionListeners(), AddBoardButtonsActionListeners(), AddCardsButtonsActionListeners(), και τις StartTurnTimer(), PlayPlayerMusic(), καθώς και διάφορες αδιάφορες πράξεις άντλησης-μεταφοράς των δεδομένων. Οπότε, με την πρώτη, θα δημιουργηθεί/εμφανιστεί ένα καινούργιο παράθυρο, εκείνο του παιχνιδιού …

… Αυτό, θα έχει το – αρχικά άδειο – ταμπλό που πάνω διαθέτει συγκεκριμένες περιοχές τοποθέτησης του κάθε πλακιδίου από την σακούλα με την χρήση κουμπιών, όλες τις κάρτες και τα πλακίδια που έχει – αρχικά κενά – ο παίκτης με την σειρά του την δεδομένη στιγμή, τρία κουμπιά: DrawTiles (Ζητούμενο: Τράβηγμα Πλακιδίων από την Σακούλα και Τοποθέτηση στις αντίστοιχες Περιοχές του Ταμπλό – αυτόματα), EndTurn που απλά δίνει/μεταβιβάζει την σειρά στον επόμενο παίκτη, και Save&Exit (Ζητούμενο: Αποθήκευση Παιχνιδιού), επίσης μία ετικέτα με το όνομα του παίκτη που έχει την σειρά του καθώς και – δίπλα – τον χρόνο που του απομένει για να παίξει τον γύρο του, και τέλος κάποιες άλλες ασήμαντες/αισθητικές λεπτομέριες. Οπότε, αφού έχουμε το παράθυρο παιχνιδιού μας, η δεύτερη επόμενες μεθόδοι που αναφέρθηκαν προσθέτουν στα νέα μας κουμπιά λειτουργικότητα. Σχόλιο: στην συγκεκριμένη φάση, βρισκόμαστε λίγο πριν αρχίσει το παιχνίδι, οπότε τώρα θα εκτελεστεί κάποιος κώδικας που αφαιρεί τέσσερα τυχαία πλακίδια από την σακούλα και τα τοποθετεί στο ταμπλό, και επιλέγει τυχαία έναν από τους τέσσερεις παίκτης για να παίξει/ξεκινήσει πρώτος. Έπειτα, οι τελευταίες μεθόδοι απλά αρχίζουν την αντίστροφη μέτρηση του γύρου – σε ένα ανεξάρτητο νήμα που συνεχώς ανανεώνετε – (Ζητούμενο: Timer), ενώ και την μουσική υπόκρουση που υπάρχει κατά την διάρκεια του παιχνιδιού και ανάλογα τον παίκτη (Ζητούμενο: Μουσική). Έτσι, αυτή την στιγμή είναι όλα έτοιμα και ο πρώτος παίκτης αρχίζει να παίζει. Οι ενέργειες που έχει στην διάθεσή του είναι οι εξής: πάτημα κουμπιού τραβήγματος πλακιδίων από σακούλα – μία φορά ανά γύρο, τέλος της σειράς του πατώντας το αντίστοιχο κουμπί, χρήση μία κάρτα (Ζητούμενο: Χρήση Καρτών Χαρακτήρα), επιλογή πλακιδίου από το ταμπλό και απόκτησή του (Ζητούμενο: Τράβηγμα Πλακιδίου από το Ταμπλό). Οι δύο τελευταίες γίνονται αφού το επιτρέψουν οι κατάλληλη έλεγχοι που γίνονται αντίστοιχα στις μεθόδους AddCardsButtonsActionListeners() και GetOneTileFromTheBoard(…) στο Model. Από την άλλη, το κουμπί τραβήγματος πλακιδίων εκτελεί επίσης αρκετά πράγματα, αλλά κυρίως τις μεθόδους DrawTilesFromBag(…) και CheckForGameOver() στο Model. Η δεύτερη, απλά ελέγχει εάν η συνθήκη τερματισμού του παιχνιδιού έχει ικανοποιηθεί (Ζητούμενο: Έλεγχος για Τέλος Παιχνιδιού). Και, το κουμπί που τερματίζει την σειρά, απλά αλλάζει/εμφανίζει όλα τα δεδομένα του επόμενου παίκτη – όπως είπαμε παραπάνω. Επιπλέον, εάν όλοι οι παίκτες συμφωνήσουν, μπορεί να πατηθεί το κοινό κουμπί αποθήκευσης παιχνιδιού, το οποίο απλά καλεί την μέθοδο SaveGameStateAndExit(…) του Model, η οποία αντιγράφει όλα τα δεδομένα της δεδομένης στιγμής σε ένα αρχείο (με όνομα “cookies.txt”) και έπειτα επιστρέφει στο αρχικό παράθυρο – μενού, επιτρέποντας την μελλοντική συνέχιση από το ίδιο ακριβώς σημείο, με την χρήση του προ αναφέροντος κουμπιού. Κατά τα άλλα, αυτή η επαναληπτική διαδικασία – με τον αντίστοιχο κώδικα - συνεχίζεται μέχρι να καταλάβει ο Controller πως τελείωσε το παιχνίδι. Σε αυτή την περίπτωση, καλούνται οι μεθόδοι GameOver(…) του View και AddGameOverScreenActionListeners() του Controller. Η πρώτη απλά αλλάζει το παράθυρο στο τελευταίο, το οποίο δείχνει τον νικητή και τους συνολικούς του πόντους, χρησιμοποιώντας την μέθοδο του Model CalculatePlayerPointsAtGameOver() (Ζητούμενα: Υπολογισμοί Πόντων – όλων), καθώς και εμφανίζει ένα κουμπί το οποίο η δεύτερη ενισχύει, και όταν πατηθεί επιστρέφουμε στο αρχικό μενού, για ακόμη μια φορά. Σχόλιο: κάθε φορά που επιστρέφουμε στο αρχικό μενού, εκτελούμε τις μεθόδους ResetModel() & ResetView() ώστε να είμαστε έτοιμοι να αρχίσουμε ένα νέο παιχνίδι. Ομοίως, λειτουργεί και το παιχνίδι για έναν παίκτη. Τέλος, να σημειωθεί πως οτιδήποτε δεν αναφέρθηκε εδώ λειτουργεί με βάση τους κανόνες αλλά και την λογική/φαντασία. Και για πληρότητα, στα αρχεία του κώδικα υπάρχουν σχόλια και τέστ για περεταίρω τεκμιρίωση/πληροφορίες/βοήθεια (Ζητούμενα: Javadoc & Junit). Έτσι, λοιπόν, λειτουργεί – σε γενικές γραμμές και χοντρικά – ο κώδικα του παιχνιδιού …

7. Συμπεράσματα και Σχόλια για την Φάση #2:

Λοιπόν, εδώ θα ήθελα να αναφέρω τα σχόλιά μου σχετικά με τη δεύτερη φάση του πρότζεκτ. Αρχικά, προσωπικά, το συγκεκριμένο πρότζεκτ μου φάνηκε αρκετά δύσκολο. Αυτό οφείλεται σε ποικίλους λόγους, όπως το ότι είναι η πρώτη φορά που πρέπει να δημιουργήσω κάτι τόσο μεγάλο και περίπλοκο σε μία γλώσσα που πρόσφατα έμαθα, όπως το ότι είναι δύσκολο να σκεφτείς και να σχεδιάσεις μόνος σου όλες τις κλάσεις και μεθόδους που χρειάζονται, όπως το ότι το μοντέλο που πρέπει να ακολουθήσουμε είναι ιδιαίτερο, ή όπως το ότι η εκφώνηση ήταν για ακόμα μια φορά απογοητευτική, κτλ …

Από την άλλη, κατάφερα να ετοιμάσω όλα τα παραδοτέα της συγκεκριμένης φάσης, τα οποία αναφέρονται αναλυτικότατα στην προηγούμενη ενότητα, και είμαι ικανοποιημένος από την επίδοσή μου, καθώς και από το αποτέλεσμά μου …

Τέλος, θα σας παρακαλούσα να ρίξετε μια ματιά στο σύντομο αλλά σημαντικό ReadMe.md αρχείο, το οποίο υπάρχει στον συγκεκριμένο φάκελο (ή στο ReadMe.txt αν δεν έχετε τρόπο για να ανοίξετε το πρώτο – παρόλο που είναι πιο ωραίο / όμορφα σχεδιασμένο) …

[…]

(◎)

[ΤΕΛΟΣ]

(ΕΥΧΑΡΙΣΤΩ)